

RoadVision

Rilevamento automatico dei danni stradali con la Computer Vision

Documento di analisi tecnica

Progetto finale di Computer Vision

Claudio La Gona

Compito	Object detection multiclasse
Dataset	RDD2022 - subset Japan
Classi target	D00, D10, D20, D40
Approcci	HOG + Linear SVM e YOLO11s

1. Descrizione del problema

Per questo progetto ho scelto di realizzare RoadVision, un sistema per rilevare automaticamente i danni visibili sulla strada a partire da immagini. L'obiettivo è localizzare ogni danno con una bounding box e assegnarlo a una delle quattro classi considerate: crepa longitudinale (D00), crepa trasversale (D10), crepa a ragnatela (D20) e buca (D40). Ho impostato il problema come object detection perché non basta riconoscere la presenza del danno: è necessario anche capire dove si trova nell'immagine. Il compito è reso più difficile dalle diverse dimensioni e forme dei danni e dalla presenza di segnaletica, ombre, vegetazione, veicoli e texture dell'asfalto.

2. Dataset e preprocessing

Ho utilizzato il subset Japan del Road Damage Dataset 2022 (RDD2022). Le annotazioni originali sono in formato Pascal VOC XML. Nella fase di preprocessing ho letto le annotazioni, controllato la geometria delle bounding box, mantenuto solo le quattro classi target, creato uno split fisso a livello di immagine e convertito le coordinate nel formato richiesto da YOLO. Durante il controllo ho trovato una sola bounding box degenerata: l'ho esclusa dai dati processati senza modificare il dataset raw originale.

Classe	Significato	Oggetti
D00	Crepa longitudinale	4,049
D10	Crepa trasversale	3,979
D20	Crepa a ragnatela	6,198
D40	Buca	2,243

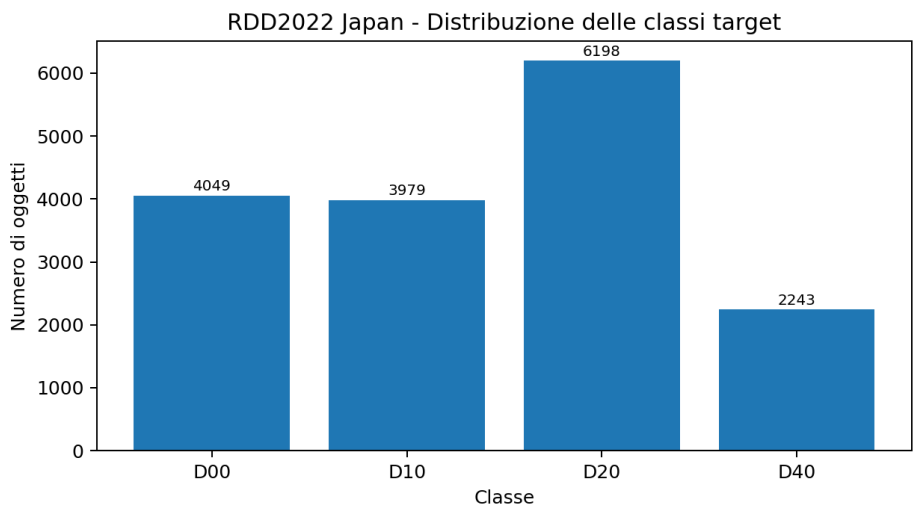


Figura 1. Distribuzione delle quattro classi target nelle annotazioni processate del subset Japan.

Ho utilizzato uno split fisso 70/15/15. Le scelte sul modello sono state fatte usando training e validation, mentre il test set è stato lasciato separato e utilizzato soltanto per la valutazione finale.

Split	Immagini	Oggetti target
Train	7,354	11,563
Validation	1,576	2,419
Test	1,576	2,487

3. Baseline classico: HOG + Linear SVM

Come baseline classico ho seguito la pipeline HOG + SVM vista durante il corso. I campioni positivi sono ottenuti ritagliando le bounding box ground truth dei danni, mentre i campioni negativi sono crop casuali presi da immagini che non contengono nessuna delle quattro classi target.

Ogni patch viene ridimensionata a 64×64 pixel e trasformata in un vettore di feature HOG, che descrive principalmente la distribuzione degli orientamenti dei gradienti. Queste feature vengono poi utilizzate per addestrare una SVM lineare in grado di classificare ogni patch come D00, D10, D20, D40 oppure background.

Sul validation set il classificatore ha ottenuto:

Accuracy: 0.7259 | Macro F1: 0.6382 | Weighted F1: 0.7148

Classe	Precisione	Recall	F1
D00	0.5659	0.3993	0.4683
D10	0.8232	0.8275	0.8254
D20	0.6195	0.6349	0.6271
D40	0.6178	0.3565	0.4521
Sfondo	0.7747	0.8669	0.8182

3.1 Detection con sliding window

Per passare dalla classificazione di singole patch alla detection sull'immagine completa, ho applicato il classificatore HOG + SVM attraverso una sliding window.

In pratica, alcune finestre di dimensione fissa scorrono sull'immagine e ogni regione viene classificata dalla SVM. Le finestre riconosciute come background vengono scartate, mentre sulle detection rimanenti viene applicata la Non-Maximum Suppression (NMS) per ridurre le bounding box sovrapposte.

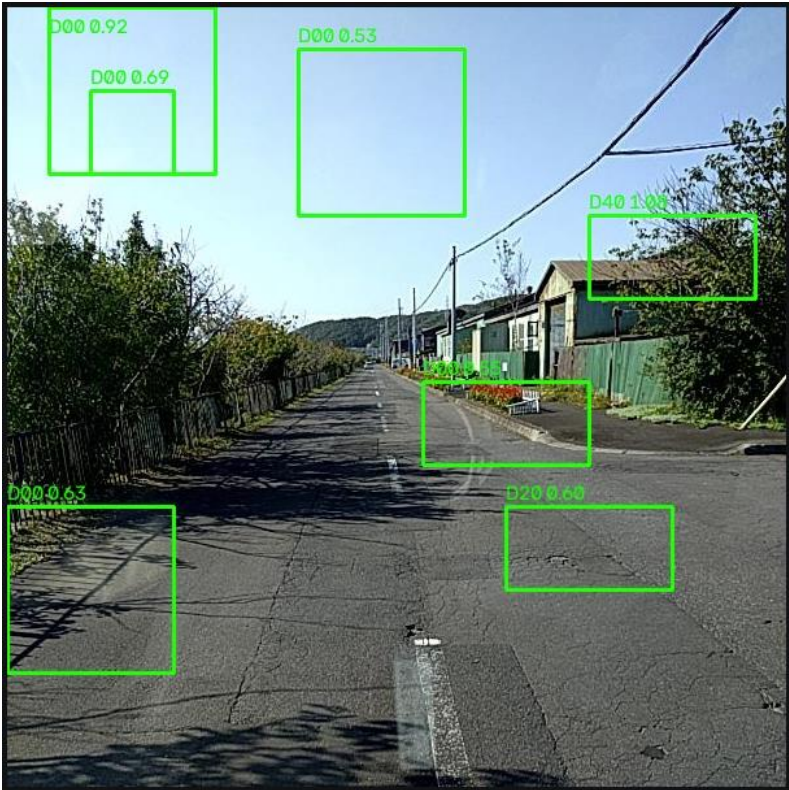


Figura 2. Esempio del detector classico con sliding window: sono presenti diversi falsi positivi e bounding box poco precise.

L'esempio mostra il principale limite che ho osservato nell'approccio classico. HOG descrive soprattutto bordi e orientamenti dei gradienti e non comprende il contenuto della scena. Di conseguenza, quando vengono analizzate molte finestre, anche ombre, vegetazione, edifici o texture dell'asfalto possono produrre pattern simili a quelli dei danni e generare falsi positivi.

Inoltre le bounding box della sliding window sono poco precise perché dipendono dalle dimensioni delle finestre scelte. Questi limiti hanno motivato il passaggio a un detector basato su Deep Learning.

4. Metodo Deep Learning: YOLO11s

Come modello principale ho scelto YOLO11s, un detector pre-addestrato che ho successivamente adattato alle quattro classi di RoadVision tramite fine-tuning.

Ho utilizzato immagini di input a 640 pixel, batch size 8 e un massimo di 50 epoche. Ho inoltre utilizzato l'early stopping con patience 10, in modo da interrompere il training quando le prestazioni sulla validation non miglioravano più.

Nel progetto finale ho mantenuto due esperimenti principali: un baseline e una seconda versione con data augmentation.

Nel primo esperimento ho utilizzato YOLO11s senza augmentation aggiuntiva, in modo da avere un punto di partenza con cui confrontare le modifiche successive.

Nel secondo esperimento ho mantenuto lo stesso modello e la stessa risoluzione, aggiungendo alcune trasformazioni semplici ai dati di training: horizontal flip, piccole traslazioni, variazioni di scala e leggere variazioni dei valori HSV.

Ho scelto la data augmentation perché il baseline mostrava una differenza tra le prestazioni di training e validation e volevo migliorare la capacità del modello di generalizzare senza aumentare la complessità dell'architettura.

Esperimento	Precisione	Recall	mAP@0.5	mAP@0.5:0.95
Baseline - Validation	0.407	0.420	0.368	0.152
Con augmentation - Validation	0.546	0.495	0.507	0.232
Finale - Test	0.538	0.488	0.490	0.219

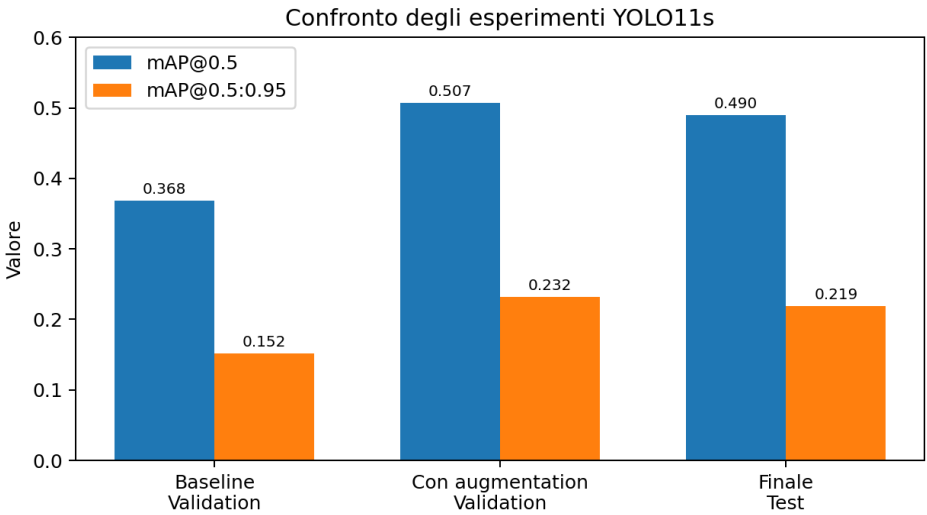


Figura 3. Confronto tra YOLO11s baseline, modello con augmentation su validation e risultato finale sul test.

La data augmentation ha migliorato in modo evidente i risultati sulla validation. La mAP@0.5 è passata da **0.368 a 0.507**, mentre la mAP@0.5:0.95 è passata da **0.152 a 0.232**.

Per questo motivo ho scelto YOLO11s a 640 pixel con data augmentation come modello finale. Dopo aver scelto il modello sulla base dei risultati di validation, ho utilizzato il test set soltanto per la valutazione finale.

5. Risultati sperimentali finali

Dopo aver scelto il modello finale usando la validation, l'ho valutato sul test set, che fino a quel momento era rimasto separato. Il modello ha ottenuto **Precision 0.538, Recall 0.488, mAP@0.5 0.490 e mAP@0.5:0.95 0.219**.

I risultati per classe sono riportati nella tabella seguente.

Classe	Precisione	Recall	mAP@0.5	mAP@0.5:0.95
D00	0.477	0.454	0.442	0.185
D10	0.526	0.385	0.382	0.153
D20	0.637	0.638	0.658	0.328
D40	0.509	0.475	0.479	0.209

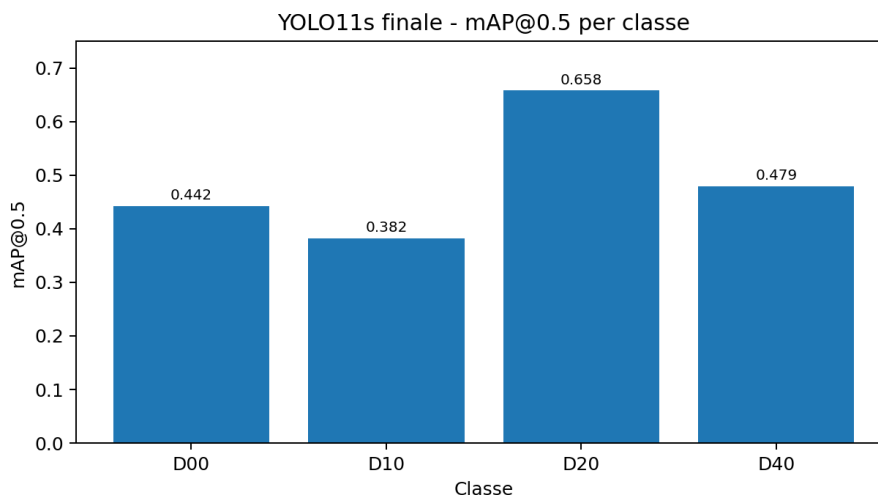


Figura 4. mAP@0.5 per classe sul test set finale.

La classe **D20** è quella con il risultato migliore, con mAP@0.5 pari a 0.658. La classe **D10** rimane invece la più difficile, soprattutto per il recall più basso e per una mAP@0.5 pari a 0.382.

Nel complesso i risultati sul test sono abbastanza vicini a quelli ottenuti sulla validation. La mAP@0.5 passa infatti da 0.507 sulla validation a 0.490 sul test. Mi aspettavo una piccola diminuzione, perché il test contiene immagini che non sono state utilizzate per scegliere il modello.

5.1 Confronto tra approccio classico e Deep Learning

Il confronto tra HOG + SVM e YOLO deve essere fatto con attenzione, perché le metriche misurano due compiti diversi. L'accuracy del modello HOG + SVM è calcolata su patch già ritagliate, mentre la mAP di YOLO valuta sia la classificazione sia la localizzazione degli oggetti nell'immagine completa.

Per questo motivo non considero corretto confrontare direttamente, ad esempio, il 72.59% di accuracy della SVM con la mAP di YOLO.

Il confronto qualitativo è però utile: HOG + SVM mi ha permesso di comprendere in modo esplicito il funzionamento delle feature handcrafted, della classificazione e della sliding window, ma sulle immagini complete produce diversi falsi positivi e bounding box poco precise. YOLO riesce invece a lavorare direttamente sull'intera immagine e produce detection molto più utilizzabili.

5.2 Demo minima su immagine

Per verificare il funzionamento del modello finale ho realizzato una demo semplice su una singola immagine. Lo script usa OpenCV per leggere l'immagine, carica il modello YOLO11s fine-tuned, esegue la detection e salva il risultato mostrando bounding box, classe e confidence.

Ho scelto volutamente una demo semplice, perché lo scopo è mostrare il funzionamento del modello su una nuova immagine senza aggiungere componenti di deployment non necessarie al progetto.



Figura 5. Output della demo finale su una singola immagine.

6. Analisi degli errori

Oltre alle metriche complessive ho osservato alcune immagini della validation in cui il modello commette errori. Ho scelto tre esempi diversi perché mi permettono di capire meglio quali situazioni risultano ancora difficili per il detector.

6.1 D10 non rilevate

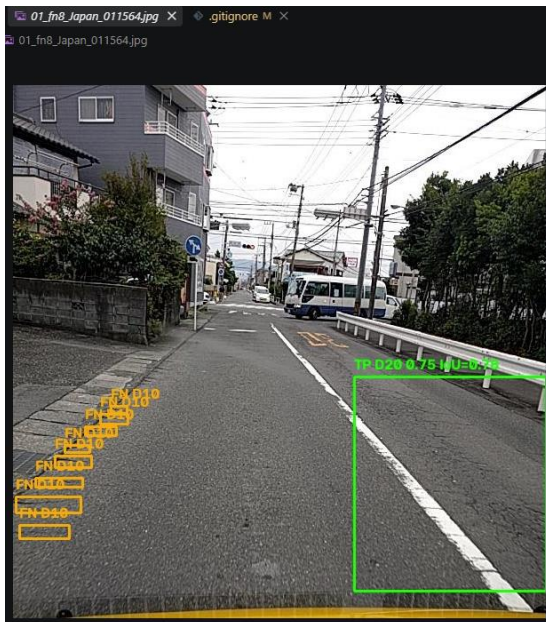


Figura 6. Esempio di crepe D10 non rilevate sulla validation.

Nel primo esempio sono presenti diverse crepe D10 che il modello non rileva. In questa immagine alcune crepe sono sottili, occupano regioni relativamente piccole e hanno poco contrasto rispetto all'asfalto.

Questo esempio è coerente con i risultati quantitativi:
D10 è infatti la classe con il recall più basso nel test finale.

6.2 Falsi positivi e localizzazione imprecisa

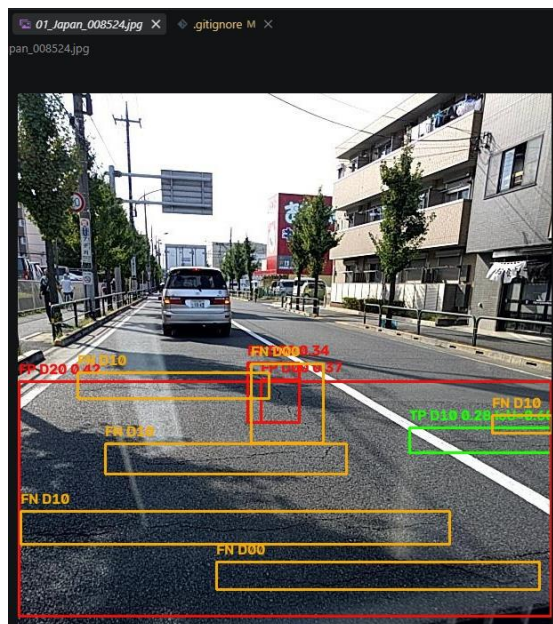


Figura 7. Esempio con falsi positivi e localizzazione imprecisa sulla validation.

Nel secondo esempio il modello produce sia detection corrette sia errori. Sono presenti falsi positivi, alcune ground truth non rilevate e una D10 individuata correttamente.

Osservando l'immagine, texture dell'asfalto, segnaletica e ombre **possono contribuire** a rendere più difficile la distinzione tra un vero danno e il background. Da una singola immagine non è possibile stabilire con certezza quale elemento abbia causato la predizione errata, ma il caso mostra che il modello può confondere pattern visivi simili.

6.3 Più danni nella stessa scena

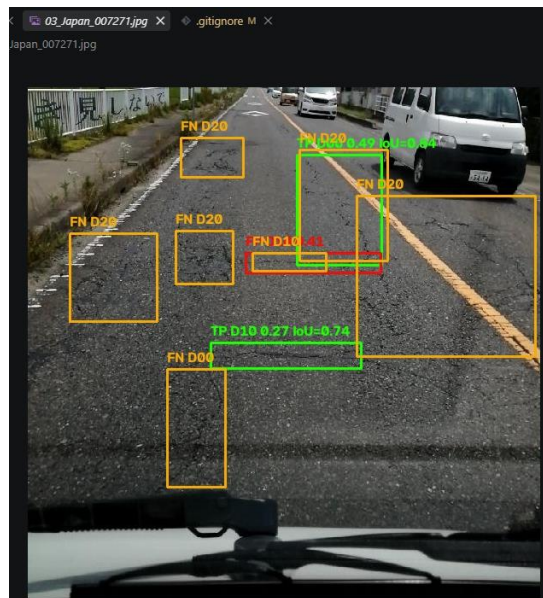


Figura 8. Scena complessa con più danni e risultati di detection differenti.

Nel terzo esempio sono presenti numerosi danni con dimensioni e forme differenti. Il modello rileva correttamente alcune regioni, ma ne perde altre e produce anche una predizione errata.

Questo caso mostra che le scene più complesse, con diversi oggetti contemporaneamente, possono essere più difficili da analizzare rispetto a immagini con pochi danni ben visibili.

Per questo motivo considero importante affiancare alle metriche aggregate anche l'osservazione diretta di alcuni errori: un singolo valore di mAP non permette di capire tutti i modi in cui il modello può fallire.

7. Considerazioni etiche

In un possibile utilizzo reale, le immagini stradali potrebbero contenere persone, veicoli e targhe. Per questo motivo la raccolta, la conservazione e l'utilizzo delle immagini devono tenere conto della privacy.

Un altro aspetto importante riguarda il bias del dataset. Il modello è stato addestrato e valutato sul subset Japan di RDD2022, quindi non posso assumere che le stesse prestazioni valgano automaticamente in altri paesi o in condizioni differenti di asfalto, illuminazione, meteo o acquisizione delle immagini.

Considererei quindi RoadVision come uno strumento di supporto e non come un sistema completamente affidabile per prendere decisioni automatiche, perché sono ancora possibili falsi positivi e danni non rilevati.

8. Conclusioni

Con RoadVision ho applicato una pipeline completa di Computer Vision a un problema reale di object detection.

Il baseline HOG + Linear SVM mi ha permesso di mettere in pratica le feature handcrafted, la classificazione, la sliding window e la Non-Maximum Suppression, osservando direttamente i limiti di questo approccio sulle immagini complete.

Successivamente ho utilizzato YOLO11s pre-addestrato e fine-tuned sulle quattro classi del progetto. Il miglioramento principale è arrivato con la data augmentation: la mAP@0.5 sulla validation è passata da 0.368 a 0.507. Il modello finale ha poi ottenuto sul test una mAP@0.5 pari a 0.490 e una mAP@0.5:0.95 pari a 0.219.

La parte più utile del progetto è stata poter confrontare concretamente un metodo classico con un approccio Deep Learning e capire l'importanza di separare training, validation e test. L'analisi degli errori mi ha inoltre mostrato che le metriche aggregate sono importanti, ma devono essere accompagnate dall'osservazione dei casi in cui il modello fallisce.